

Meeting Notes

- Establish a dedicated meeting log for the project and record the following details for every meeting:
 - **Date:** The date of the meeting
 - **Time:** The start and end times of the meeting
 - **Location:** The physical or virtual location of the meeting
 - **Objective:** The main objective or agenda of the meeting
 - **Team members:** List of all team members present at the meeting
 - **Tasks allocated:** List specific tasks assigned to each team member, including deadlines
 - **Task completion confirmation:** Starting from the second meeting, confirm whether each team member completed the tasks allocated to them in previous meetings

Meeting 1 Notes

Meeting 1 Notes

- **Date:** (9/10/2026)
- **Time:** 4:00-5:00
- **Location:** LEEP 2324
- **Objective:** divvy up roles/responsibilities
- **Team members:** everyone
- **Tasks allocated:**

1. Board, Documentation & System Architecture Diagram → Emma R

getCell(row, col)
setCell(row, col)
reset()

Manages the underlying 10×10 grid data structure, tracking the state of each cell (covered, flagged, uncovered, or mine). It provides clean getter and setter methods (`getCell`, `setCell`) so other modules can inspect or mutate cell values without directly manipulating raw arrays. Its `reset()` method clears and reinitializes the entire board to a blank state for a new game.

2. MineGenerator → Arpa

generateMines(board, mineCount, safeCell)

Handles the randomized distribution of the user-specified 10–20 mines across the 10×10 grid. It takes the player's initial clicked cell (`safeCell`) as a parameter to guarantee that the first move never hits a mine (and optionally keeps adjacent cells safe). It updates the `Board` data structure with the placed mine coordinates once generation is complete.

3. NumberCalculator → Serom

calculateNumbers(board)

Iterates through every non-mine cell on the board to compute the count of adjacent mines (0–8) within its 8 neighboring cells. It executes immediately after mine generation so that each cell stores its correct numerical value before gameplay starts. It writes these calculated adjacency values back into the `Board` model for quick lookup during cell reveals.

4. Game → Jana & Ellie

uncover(row, col)
toggleFlag(row, col)
getStatus()

Orchestrates overall gameplay rules, including tracking remaining flags, triggering recursive flood-fill revealing when a player uncovers a zero-value cell, and toggling flag states. It evaluates win/loss conditions on every move (`getStatus`) to determine if a mine was detonated or if all non-mine cells have been successfully uncovered. It also ensures flagged cells cannot be accidentally uncovered until unflagged.

5. UI → Megan

`render(board, gameState)`

Renders the 10×10 visual grid with columns labeled A–J and rows numbered 1–10, showing each cell's current visual state (covered, flagged, uncovered number, or blank). It displays game metadata, including remaining flag count (`total mines - flags placed`) and game status messages ("Playing", "Victory", "Game Over"). Upon a loss, it handles revealing the locations of all detonated mines to the player.

6. InputHandler → sneha and emma du

`handleClick(row, col)`

`handleFlag(row, col)`

|
|
|
v

Just combine into `InputHandler(event)`

Captures and parses player inputs (such as left-clicks for revealing cells and right-clicks or key modifiers for toggling flags) from the interface. It validates coordinate boundaries (A–J, 1–10) and checks that the action is legal before forwarding the request to the `Game` controller. It ensures seamless communication between the front-end user actions and back-end state updates.

Forward request to the game controller in the main game loop, not inside `inputHandler`.

minesweeper/

- ├── docs/
 - ├── Architecture.png # 20 pts: Component diagrams, data flow, data structures
 - └── Minesweeper.pdf # Documentation logs detailing who worked on what/for how long
- ├── src/
 - ├── __init__.py
 - ├── cell.py # Low-level cell state model
 - ├── board.py # Board Manager (Emma R)
 - ├── mine_generator.py # MineGenerator (Arpa)
 - ├── number_calculator.py # NumberCalculator (Serom)
 - ├── game.py # Game Logic (Jana & Ellie)
 - ├── ui.py # User Interface (Megan)
 - ├── input_handler.py # Input Handler (Sneha & Emma Du)
 - └── main.py # Application entry point
- ├── tests/ # Unit and stress tests
 - ├── test_board.py
 - ├── test_game.py
 - ├── test_number_calculator.py
 - └── test_cell.py
- ├── README.md # Setup & run instructions
- └── requirements.txt # Dependencies (if any)

Complete Documentation

Name	Task	Estimated hours	Actual hours	Date
All members of the group	Meet and allot responsibilities	1	1.5	9/7
Emma D. and Sneha	Write input_handler.py	1.5	2	9/7
Megan	Write ui.py	2	3	9/10, 9/16
Serom	Write cell.py	1.5	1	9/14
Serom	Write number_calculator.py	1.5	2	9/16
Arpa	Write mine_generator.py	2	1	9/16
Jana and Ellie	Write game.py	2	2	9/16
Serom	wrote test files for cell and number_calculator.py	2	1	9/16
Emma R.	Write board.py Write Documentation	3	3	9/14,9/15,9/16
All members of the group	Met up to help connect all file functionalities & frontend/backend, went over any roadblocks/problems met.	2	3	9/17

Date: September 7th

Time: 3PM

Location: LEEP2 2324

Duration: 2 Hours

Objective: Setup the repository and the file directory so that all members can pull the exact same working directory. Divide tasks and files among each member to ensure all aspects of both frontend and backend have been assigned. Discuss basic implementation details to ensure everyone is on the same page and begin basic implementation for each file.

Team Members Present: Megan, Sneha, Emma D., Arpa, Serom, Emma R., Jana, Ellie

Task Completion Confirmation:

- Emma D.: Initialized the repository and set up a basic working file directory to be initialized within everyone's local computer.
- Everyone: Discuss basic file implementations to ensure all tasks and requirements are being completed by the appropriate member.
- Everyone: Begin basic file function implementations within their respective files to ensure that everyone has a good starting point to then work on each file by themselves.

Date: September 7th

Time: 5PM

Location: Zoom

Duration: 1 Hour

Objective: Created a class that checks for two types of events and each event is a mouse click. The first event is if it's a left click while the second event is the middle click. What it returns is a dictionary that lists what kind of action (flag or unflag) and what coordinates you clicked and sends that information to [game.py](#)

Team Members Present: Sneha, Emma D.

Task Completion:

- Created a class that checks for two types of events and each event is a mouse click.
 - The first event is if it's a left mouse click or middle mouse click. What it returns is a dictionary that lists what kind of action (flag or unflag) and what coordinates you clicked and sends that information to [game.py](#)
 - Sneha & Emma D.: Researched the correct implementation outline such that the appropriate reaction will be outputted based on mouse activity
 - Sneha & Emma D. : Finished the input_handler.py file
-

Date: September 10th, September 16th

Time: 6PM, 2PM

Location: At home

Duration: 3 Hours

Objective: Create a UI for the board so that minesweeper has a grid to be played on. Also create icons (flag, bomb, grid squares) to be utilized by the user for an enjoyable game experience.

Team Member Present: Megan Svoren

Task Completion:

- Megan: Hand drawn icons were completed and imported within the file directory.
 - Megan: Created the 10x10 grid and implemented the UI functionality for this grid.
 - Megan: Full completed the [UI.py](#) file.
-

Date: September 14th, September 16th

Time: 7PM, 3PM,

Location: LEEP2, At home

Duration: 4 Hours

Objective: For [Cell.py](#), created and defined a cell class to keep track of current visible state (if it has a mine, and number of adjacent mines). For [number_calculator.py](#) calculates the number of adjacent mines for every non-mine cell. Stores this number within each corresponding cell object. For test files, created two files to test [cell.py](#) functionality and [number_calculator.py](#)

Team Member Present: Serom Kim

Task Completion:

- Serom: Defined the Cell class used to represent an individual cell on the board.
 - Serom: Calculated the number of adjacent mines for every non-mine cell on the board. The calculated values 0 to 8, are stored in each Cell object for quick access during gameplay.
-

Date: September 16th

Time: 3PM

Location: LEEP2 2326

Duration 1 Hour

Objective: Randomly places the chosen number of mines (10-20) on the 10x10 board, making sure the player's first-clicked cell never gets a mine.

Team Members Present: Arpa Das

Task Completion:

- Arpa: Defined the MineGenerator class used to randomly place mines on the board before the user begins playing the game.
 - Arpa: Finished the complete implementation of the mine_generator.py file, ensuring that the appropriate number specified in mineCount is displayed on the board.
-

Date: September 16th

Time: 4PM

Location: Live in the same House

Duration: 2 Hours

Objective: Implements the overall gameplay rules of minesweeper (uncover and flag) and evaluates win/loss

Team members Present: Jana Frady, Ellie Thach

Task Completion:

- Jana: Defined the GameStatus and Game class used to implement the rules of minesweeper like uncovering squares and toggling flags
 - Ellie: Defined the getStatus and remainingFlags methods as well as the private methods used get the status of the game and the number of remaining flags.
 - Jana & Ellie: Teamwork makes the dream work!!! 😁
-

Date: September 14th, September 15th

Time: 3PM

Location: Tous Les Jours, At Home

Duration: 3 Hours

Objective: Manages the backend logic for the 10x10 Minesweeper board. The board is stored as a 2D array of Cell objects. In addition, kept track and wrote out all documentation to ensure organization of this project.

Wrote various test cases to test each file and ensure everything was working correctly.

Task Completion:

- Emma: Implement the reset, getCell, setCell, and getBoard, all backend logic for the UI of the board.
 - Emma: Completed the Implementation of the entire [board.py](#) file
 - Emma: Kept track and wrote all documentation for this project.
 - Emma: Wrote test cases for each file to ensure full functionality.
-

Date: September 17th

Time: 3PM

Location: LEEP2 2324

Duration: 2 Hours

Objective: Walk through each individual file and ensure everything could connect with one another (importing functions, connecting frontend/backend), communicate any and all roadblocks so that we could discuss possible manners to overcome these challenges. Fixed up small details (comments, connections, documentation, etc.) to get the project ready for submission

Team Members Present: Megan, Sneha, Emma D., Arpa, Serom, Emma R., Jana, Ellie

Task Completion:

- Megan: Fixed the UI (any bugs, defects present), understood connection between [UI.py](#) and other files within the directory, updated code to reflect these changes.
 - Emma D: Worked with Sneha to fix defects in the Input Handler file, and ensure that the file was now fully functional. Worked with Megan to help fix various aspects of the UI.
 - Arpa: Helped Emma R. with documentation and organizing the file directories for the project to ensure everything was ready for submission.
 - Serom: Helped provide various bug fixes to teammates facing roadblocks, provided QA support for files with hidden issues/bugs.
 - Emma R: Finished up all documentation and added more details to ensure everything was recorded in an organized manner, helped connect various files that were previously not implemented within the directory.
 - Jana: Fixed importation errors in various files to ensure full functionality, worked with Ellie to improve Gameplay experience.
 - Ellie: Worked with Jana to enhance [game.py](#) functionality and provided assistance during conversations of various roadblocks.
 - Sneha: Worked with Emma D. to fix defects in the Input Handler file, and ensure that the file was now fully functional.
-